PROJECT REPORT

Secure, Autoscaling Three-Tier App on Kubernetes

FoodExpress: Online Food Ordering and Delivery Platform

Docker | Kubernetes | Helm | Calico NetworkPolicies | HPA | RBAC | k6

Three-tier application on a local Minikube cluster

React + Nginx | Python Flask | PostgreSQL | Helm | Calico | Metrics Server | k6

PREPARED BY

Parikshit Arer

DATE

November 2026

ENVIRONMENT

Minikube profile foodexpress-netpol | Namespace devops-platform | WSL Ubuntu

Abstract

FoodExpress is a containerized online food-ordering platform deployed on a Kubernetes cluster. The project shows how a multi-tier application can be packaged, deployed, secured, scaled and load-tested using Docker, Kubernetes, Helm and supporting tools. The application has a React frontend served by Nginx, a Python Flask REST API and a PostgreSQL database run as a StatefulSet with persistent storage.

The environment is a dedicated Minikube profile, `foodexpress-netpol`, running Kubernetes v1.35.1 with the Calico CNI. The backend is released through a Helm chart that also manages a Horizontal Pod Autoscaler, a Calico-enforced NetworkPolicy and namespace RBAC. Metrics Server supplies pod resource metrics.

Two tests validate the platform. A CPU stress test pushed backend utilization to about 233% of its request and the HPA scaled the deployment from 2 to 5 replicas with no pod restarts. A k6 load test of the backend health endpoint ramped to 20 virtual users and completed 1,335 requests with a 7.85 ms p95 response time and no failures. The report also records the troubleshooting steps and the limits of the current scope.

Project at a Glance

Application	FoodExpress: online food ordering and delivery (React frontend, Flask API, PostgreSQL)
Kubernetes platform	Minikube profile foodexpress-netpol, Docker driver, Kubernetes v1.35.1, Calico CNI
Namespace	devops-platform
Images	food-ordering-api:v3 (backend), food-ordering-frontend:v1 (frontend)
Package management	Helm chart foodexpress 0.1.0, release food-api
Database	PostgreSQL StatefulSet food-db with a 1Gi PersistentVolumeClaim (Bound)
Autoscaling	HPA: min 2, max 5, 70% CPU target. Observed scale-up from 2 to 5 replicas at 233% CPU
Security	Calico NetworkPolicy (ingress), RBAC Role and RoleBinding, ConfigMap and Secret
Performance test	k6, up to 20 VUs: 1,335 requests, p95 7.85 ms, 0.00% failures

Table of Contents

Abstract 2

Project at a Glance . . . 2

1. Project Overview 5

1.1 Introduction . . . 5

1.2 Project Objectives . . . 5

1.3 Technology Stack . . . 6

1.4 Scope . . . 6

2. Application Architecture 7

2.1 Three-Tier Overview . . . 7

2.2 Frontend Layer . . . 7

2.3 Backend Layer . . . 8

2.4 Database Layer . . . 8

2.5 Kubernetes Resource Inventory . . . 8

2.6 Key Design Decisions . . . 9

3. Environment and Containerization 10

3.1 Dedicated Minikube Cluster . . . 10

3.2 Docker Containerization . . . 10

3.3 Kubernetes Namespace . . . 11

4. Kubernetes Implementation 12

4.1 PostgreSQL StatefulSet and Persistent Storage . . . 12

4.2 ConfigMaps and Secrets . . . 12

4.3 Backend Deployment with Helm . . . 12

4.4 Frontend Deployment . . . 13

4.5 Horizontal Pod Autoscaler . . . 13

4.6 Network Security with Calico . . . 14

4.7 Kubernetes RBAC . . . 14

5. Deployment Verification and Results 15

5.1 Application Running . . . 15

5.2 Cluster and Workloads . . . 16

5.3 Container Image and Frontend Manifest . . . 16

5.4 Backend Helm Release . . . 17

5.5 Storage, Release, Autoscaler and Calico . . . 18

5.6 Frontend to Backend Connectivity . . . 18

5.7 Autoscaling Test . . . 19

5.8 NetworkPolicy, RBAC and Resource Usage 20

6. Performance Testing with k6 **22**

6.1 Test Setup 22
6.2 Test Script 22
6.3 Load Profile 22
6.4 Results 23
6.5 Threshold Results 24
6.6 Conclusion and Limitation 24

7. Troubleshooting **25**

7.1 Frontend Could Not Reach the Backend 25
7.2 k6 Connection Refused 26

8. Security Considerations and Implementation Summary **27**

8.1 Security Controls 27
8.2 Implementation Summary 27

9. Limitations and Future Improvements **28**

9.1 Current Limitations 28
9.2 Future Improvements 28

10. Conclusion **29**

Appendix A: Project Directory Structure **30**

Appendix B: Useful Management Commands 30

1. Project Overview

1.1 Introduction

FoodExpress is a containerized online food-ordering application deployed on a Kubernetes cluster. The project demonstrates how a multi-tier application can be deployed, managed, secured, monitored for resource utilization, and tested under simulated user load.

The application has three main components: a **React** frontend served through Nginx, a **Python Flask** REST API, and a **PostgreSQL** database deployed as a Kubernetes StatefulSet. Infrastructure is managed with Kubernetes manifests and Helm. Additional capabilities include Horizontal Pod Autoscaling (HPA), Role-Based Access Control (RBAC), Calico NetworkPolicies and Metrics Server. Performance testing with k6 evaluates the backend health endpoint under simulated concurrent traffic.

1.2 Project Objectives

1. Containerize the frontend and backend applications.
2. Deploy a multi-tier application on Kubernetes.
3. Manage application deployments using Helm.
4. Deploy PostgreSQL using a StatefulSet and persistent storage.
5. Implement automatic backend scaling using HPA.
6. Restrict backend network access using NetworkPolicies.
7. Configure Kubernetes RBAC permissions.
8. Monitor pod resource utilization.
9. Perform performance testing using k6.
10. Document the infrastructure and deployment process.

1.3 Technology Stack

Technology	Purpose
Docker	Containerization
Kubernetes	Container orchestration
Minikube	Local Kubernetes environment
React	Frontend application
Nginx	Frontend web server
Python and Flask	Backend language and REST API framework
PostgreSQL	Relational database
Helm	Kubernetes package management
Calico	Container networking and network policies
HPA	Automatic pod scaling
Metrics Server	Pod resource metrics
RBAC	Kubernetes access control
k6	Performance and load testing
Linux / WSL Ubuntu	Development environment
Git	Version control

1.4 Scope

The project covers containerization, multi-tier deployment on Kubernetes, Helm-based release management, a stateful database with persistent storage, configuration and secret management, autoscaling, network policy enforcement, RBAC, resource monitoring and load testing.

Scope boundary
The platform runs on a **local Minikube cluster**. It demonstrates core Kubernetes and DevOps practices but is not a production deployment. The persistent volume is local to Minikube and is not a backup or disaster-recovery solution.

2. Application Architecture

2.1 Three-Tier Overview

The application follows a three-tier architecture. Users reach the React frontend through the frontend Service. The frontend talks to the Flask backend through the internal `food-api-service`, and the backend talks to PostgreSQL through `food-db-service`. The database stores its data on a PersistentVolumeClaim.

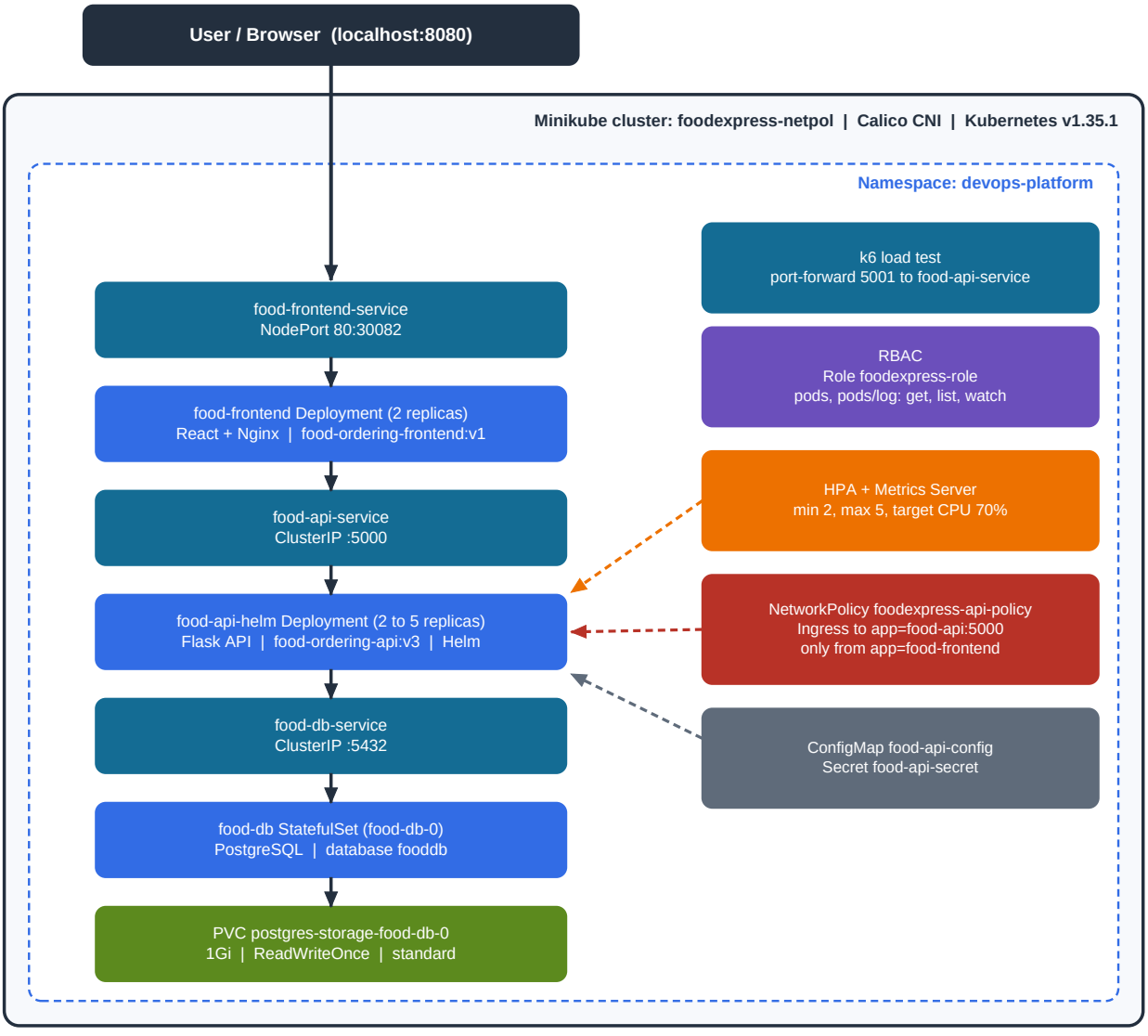


Figure 2.1: FoodExpress architecture inside the foodexpress-netpol Minikube cluster

2.2 Frontend Layer

The frontend is built with React and served through Nginx. It provides the user interface, communicates with the backend API, runs in Kubernetes pods and is exposed through a Kubernetes Service. The application home page shows the menu, the cart and the status of the backend connection.

2.3 Backend Layer

The backend is a Python Flask REST API. It handles API requests, provides the health check, communicates with PostgreSQL and reads its settings from Kubernetes ConfigMaps and Secrets. It is deployed with Helm. The key endpoint used for health checks and load testing is:

```
GET /health        ->  healthy
```

2.4 Database Layer

PostgreSQL is deployed as a StatefulSet. It stores application data, has a stable pod identity (`food-db-0`), uses persistent storage through a PersistentVolumeClaim and is reached through an internal Service.

Database	fooddb
Database username	foodadmin
Database Service	food-db-service
Database port	5432
Storage request	1Gi

Credentials are provided through a Kubernetes Secret. Passwords are intentionally excluded from this report.

2.5 Kubernetes Resource Inventory

Layer	Resource	Name	Details
Namespace	Namespace	devops-platform	Isolates all FoodExpress resources
Database	StatefulSet / Pod	food-db / food-db-0	PostgreSQL, 1/1 Ready
Database	Service	food-db-service	ClusterIP, port 5432
Database	PVC	postgres-storage-food-db-0	1Gi, ReadWriteOnce, class standard, Bound
Backend	Deployment (Helm)	food-api-helm	2 replicas, food-ordering-api:v3
Backend	Service	food-api-service	ClusterIP, port 5000
Backend	ConfigMap / Secret	food-api-config / food-api-secret	Settings and credentials
Backend	HPA	food-api-helm	min 2, max 5, 70% CPU
Security	NetworkPolicy	foodexpress-api-policy	Ingress from app=food-frontend on 5000/TCP
Security	Role / RoleBinding	foodexpress-role / foodexpress-rolebinding	Read access to pods and pod logs
Frontend	Deployment	food-frontend	2 replicas, food-ordering-frontend:v1
Frontend	Service	food-frontend-service	NodePort, 80:30082

2.6 Key Design Decisions

Decision	Rationale
Separate Minikube profile with Calico	The original cluster had no NetworkPolicy implementation. A new profile enforces policies and keeps the original cluster untouched.
Database as a StatefulSet	Gives the database a stable identity and ties it to persistent storage.
Helm for the backend	Deployment, HPA, NetworkPolicy and RBAC are installed, upgraded and removed as one release.
ConfigMap and Secret	Keeps configuration and credentials out of the container image.
Locally loaded images	minikube image load avoids needing a public container registry.
HPA on CPU	Lets the backend add replicas under load without manual action.

3. Environment and Containerization

3.1 Dedicated Minikube Cluster

To preserve the original working environment, a separate Minikube profile was created for the Calico-enabled deployment. The original profile was kept unchanged.

Minikube profile	foodexpress-netpol
Driver	Docker
Kubernetes version	v1.35.1
CNI	Calico
Namespace	devops-platform
Cluster IP	192.168.58.2

```
minikube start -p foodexpress-netpol
minikube profile list
kubectl --context foodexpress-netpol get nodes
kubectl --context foodexpress-netpol get pods -n kube-system
```

Calico components (`calico-node` and `calico-kube-controllers`) were deployed and verified as Running.

3.2 Docker Containerization

Docker packages each application component into an image. Two images are used:

- **Backend:** `food-ordering-api:v3` (Python Flask, container port 5000)
- **Frontend:** `food-ordering-frontend:v1` (React build served by Nginx, container port 80)

The frontend Dockerfile uses a multi-stage build: a Node 22 Alpine stage installs dependencies and builds the React application, and a small Nginx Alpine stage serves the build output with a custom `nginx.conf`. This keeps build tooling out of the runtime image.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Condensed from the frontend Dockerfile shown in Figure 5.4.

The images are loaded into the dedicated cluster so Minikube can use them without pulling from a public registry:

```
minikube image load -p foodexpress-netpol food-ordering-api:v3
minikube image load -p foodexpress-netpol food-ordering-frontend:v1
minikube image ls -p foodexpress-netpol
```

3.3 Kubernetes Namespace

A dedicated namespace isolates the application resources. Every FoodExpress resource lives in `devops-platform`.

```
kubectl --context foodexpress-netpol create namespace devops-platform
kubectl --context foodexpress-netpol get namespaces
```

4. Kubernetes Implementation

4.1 PostgreSQL StatefulSet and Persistent Storage

PostgreSQL runs as a StatefulSet because stateful workloads need a stable pod identity and persistent storage. The StatefulSet `food-db` creates the pod `food-db-0`, which is reached through `food-db-service`. Its PersistentVolumeClaim is `postgres-storage-food-db-0`.

Storage request	1Gi
Access mode	ReadWriteOnce
Storage class	standard
PVC status	Bound

```
kubectl --context foodexpress-netpol apply -f database/postgres.yml
kubectl --context foodexpress-netpol get statefulsets -n devops-platform
kubectl --context foodexpress-netpol get pvc -n devops-platform
```

Storage note
The PVC provides persistence inside this local Minikube environment. It is not a production-grade database backup or disaster-recovery solution.

4.2 ConfigMaps and Secrets

ConfigMaps and Secrets separate configuration from the container images. The ConfigMap `food-api-config` holds non-sensitive settings, including database connection settings. The Secret `food-api-secret` holds the database credentials.

```
kubectl --context foodexpress-netpol apply -f backend/configmap.yml -f backend/secret.yml
kubectl --context foodexpress-netpol get configmaps -n devops-platform
kubectl --context foodexpress-netpol get secrets -n devops-platform
```

Secret values must never be committed to GitHub or written into documentation.

4.3 Backend Deployment with Helm

Helm packages and manages the backend resources. The chart `helm/foodexpress` contains the Deployment, HPA, Service, ServiceAccount, NetworkPolicy, Role and RoleBinding templates, plus optional Ingress and HTTPRoute templates.

```
helm/
`-- foodexpress/
    |-- Chart.yaml
    |-- values.yaml
    `-- templates/
        |-- deployment.yaml     |-- hpa.yaml
        |-- service.yaml        |-- serviceaccount.yaml
        |-- networkpolicy.yaml  |-- role.yaml
        |-- rolebinding.yaml    |-- ingress.yaml
        |-- httproute.yaml      `-- _helpers.tpl
```

Release name	food-api
Chart	foodexpress (version 0.1.0, app version 1.16.0)
Namespace	devops-platform
Revision / status	1 / deployed
Backend image	food-ordering-api:v3, container port 5000
Rollout strategy	RollingUpdate, 25% max unavailable, 25% max surge

```
helm upgrade --install food-api helm/foodexpress \
  --kube-context foodexpress-netpol -n devops-platform
helm list --kube-context foodexpress-netpol -n devops-platform
```

4.4 Frontend Deployment

The React frontend is deployed with a Deployment and a Service manifest. Both replicas reach the Running state. Requests and limits keep the pods small and predictable, and readiness and liveness probes check the Nginx root path.

Deployment / Service	food-frontend / food-frontend-service (NodePort 80:30082)
Replicas	2
Image	food-ordering-frontend:v1 (imagePullPolicy IfNotPresent)
Resources	Requests 100m CPU, 64Mi memory. Limits 300m CPU, 128Mi memory
Probes	Readiness and liveness: HTTP GET / on port 80

```
kubectl --context foodexpress-netpol apply -f frontend/deployment.yml -f frontend/service.yml
kubectl --context foodexpress-netpol port-forward svc/food-frontend-service 8080:80 -n devops-platform
# open http://localhost:8080
```

The frontend reaches the backend through the internal Service `food-api-service` on port 5000.

4.5 Horizontal Pod Autoscaler

The HPA is installed by the Helm chart. It adjusts the number of backend replicas according to observed CPU utilization measured against the pod CPU request. Metrics Server must be running for the HPA to read CPU metrics.

Setting	Value
Minimum replicas	2
Maximum replicas	5
Target CPU utilization	70%
CPU request / limit	100m / 500m
Memory request / limit	128Mi / 256Mi

```
minikube addons enable metrics-server -p foodexpress-netpol
kubectl --context foodexpress-netpol top pods -n devops-platform
kubectl --context foodexpress-netpol get hpa -n devops-platform
```

4.6 Network Security with Calico

Calico is the Container Network Interface for the dedicated cluster and enforces NetworkPolicies. The components `calico-node` and `calico-kube-controllers` were verified as Running. The Helm chart deploys the NetworkPolicy `foodexpress-api-policy`.

Pods protected	app=food-api (backend)
Allowed ingress	TCP port 5000 from pods labelled app=food-frontend
Policy type	Ingress only. Egress is not restricted

The policy limits backend ingress to the frontend pods on the application port, which reduces unnecessary network access to the backend.

Important
This policy is an ingress restriction, not a complete zero-trust design. Further policies would be needed to restrict egress and other traffic paths.

4.7 Kubernetes RBAC

RBAC manages permissions inside the application namespace. The Role `foodexpress-role` grants read access, and the RoleBinding `foodexpress-rolebinding` attaches it to the `default` ServiceAccount in `devops-platform`.

API group	Resource	Verbs
Core	pods	get, list, watch
Core	pods/log	get, list, watch

```
kubectl --context foodexpress-netpol auth can-i get pods \
  --as=system:serviceaccount:devops-platform:default -n devops-platform
```

Improvement opportunity
The RoleBinding uses the namespace default ServiceAccount. A dedicated ServiceAccount with only the required permissions would give better isolation.

5. Deployment Verification and Results

This chapter presents terminal and browser evidence that the platform was deployed and behaves as designed. Figures come from the dedicated `foodexpress-netpol` cluster unless a caption says otherwise.

5.1 Application Running

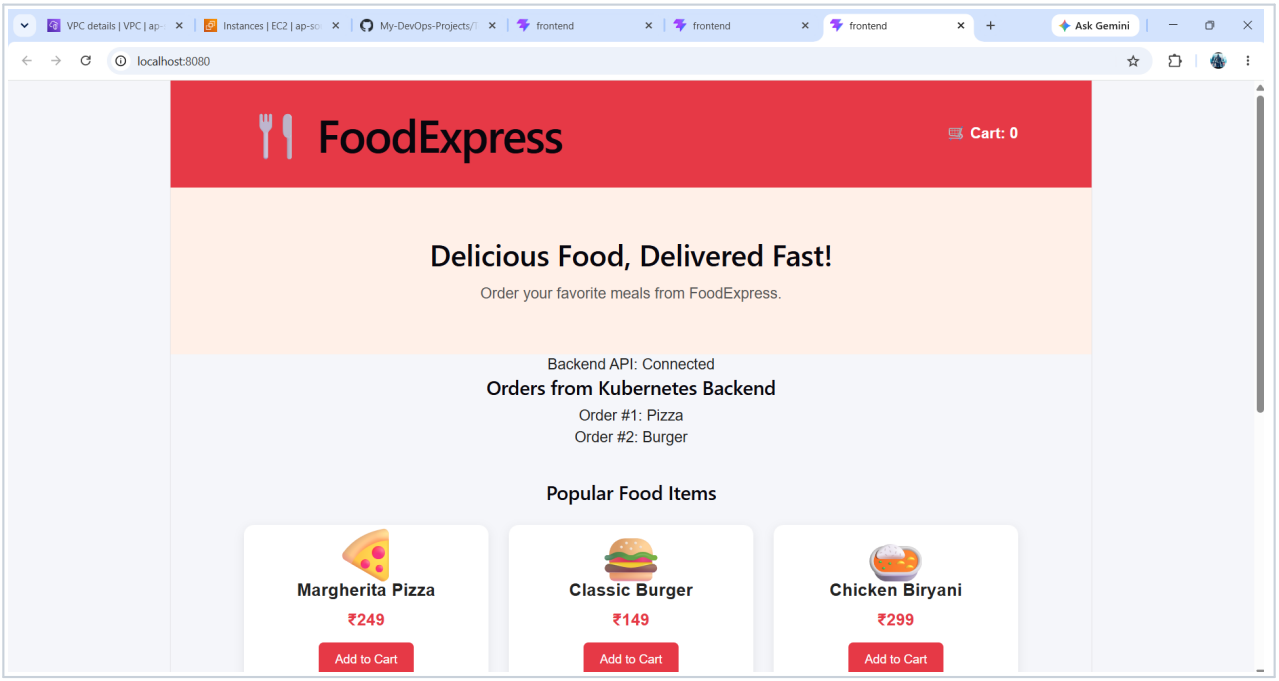


Figure 5.1: FoodExpress frontend at localhost:8080 showing Backend API: Connected and the orders returned by the Kubernetes backend.

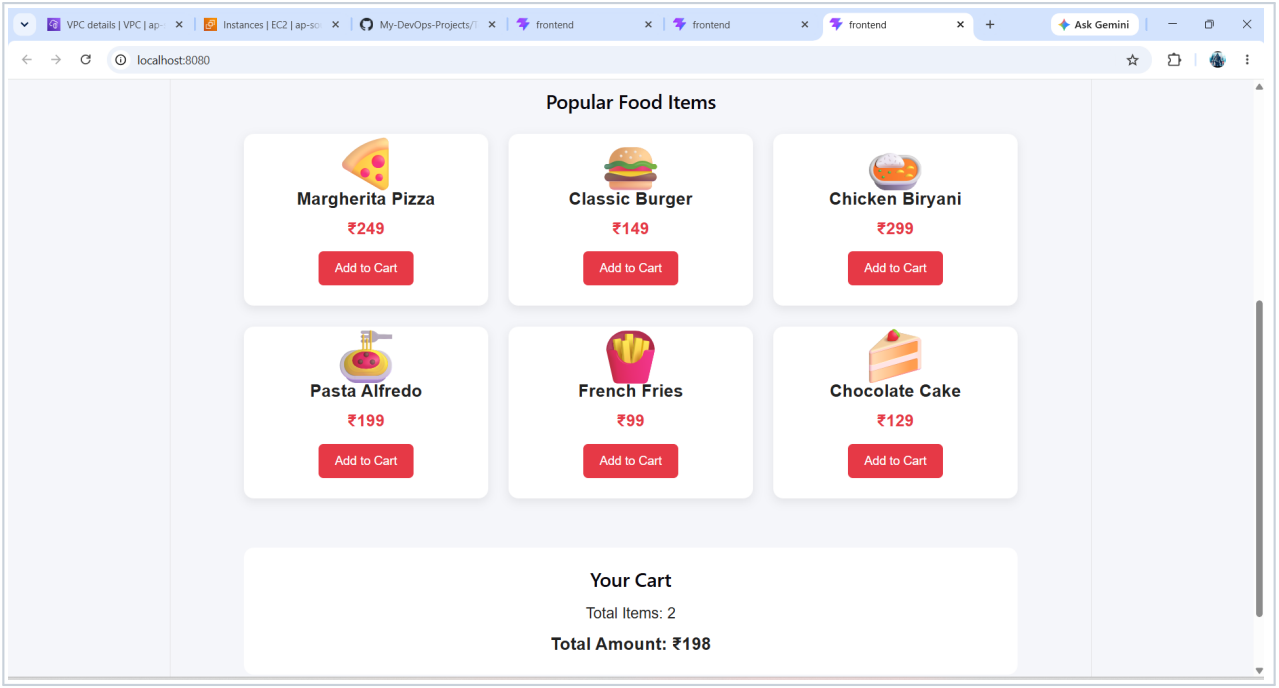


Figure 5.2: Menu items with prices and a working cart (2 items, total 198 rupees).

The status line and the order list on the home page come from the Flask API, so the page confirms the full path from frontend to backend inside the cluster.

5.2 Cluster and Workloads

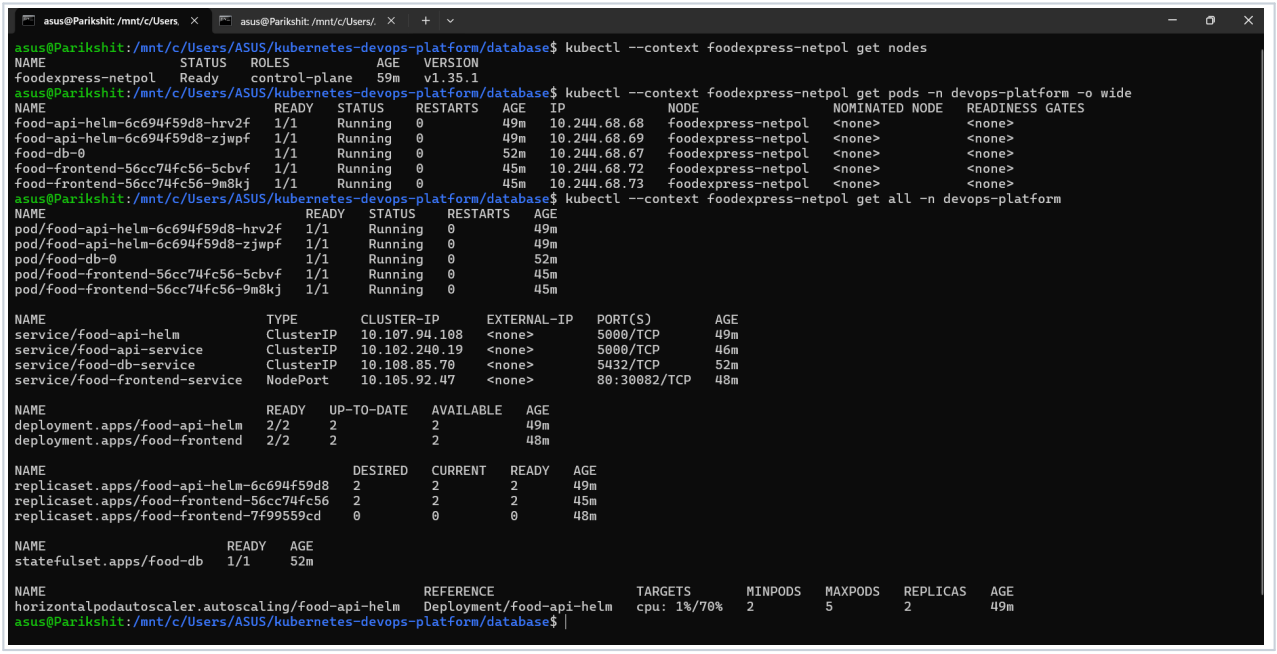


Figure 5.3: Single control-plane node (v1.35.1, Ready), all pods Running with 0 restarts, Services, Deployments, StatefulSet and the HPA.

The node is Ready on Kubernetes v1.35.1. The backend (`food-api-helm`) and frontend Deployments are both 2/2 available, the `food-db` StatefulSet is 1/1, and the frontend Service is a NodePort (80:30082). An older frontend ReplicaSet shows 0 pods, which is the result of the rolling restart performed during troubleshooting.

5.3 Container Image and Frontend Manifest

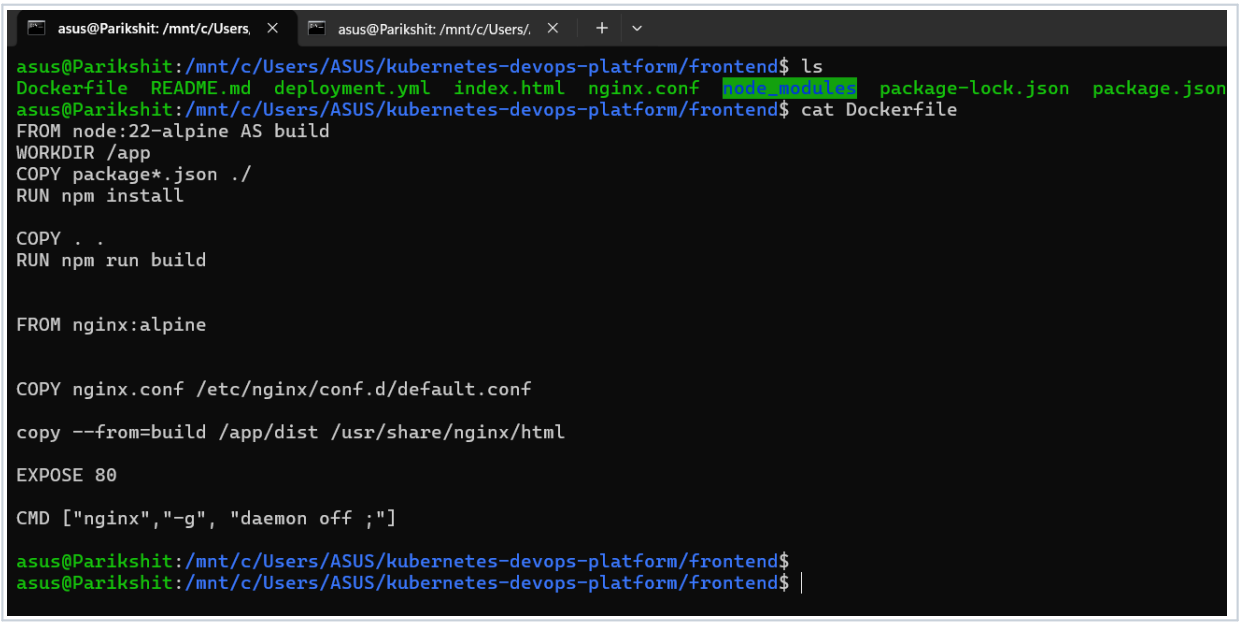


Figure 5.4: Frontend Dockerfile: multi-stage build (Node 22 Alpine) with an Nginx Alpine runtime.

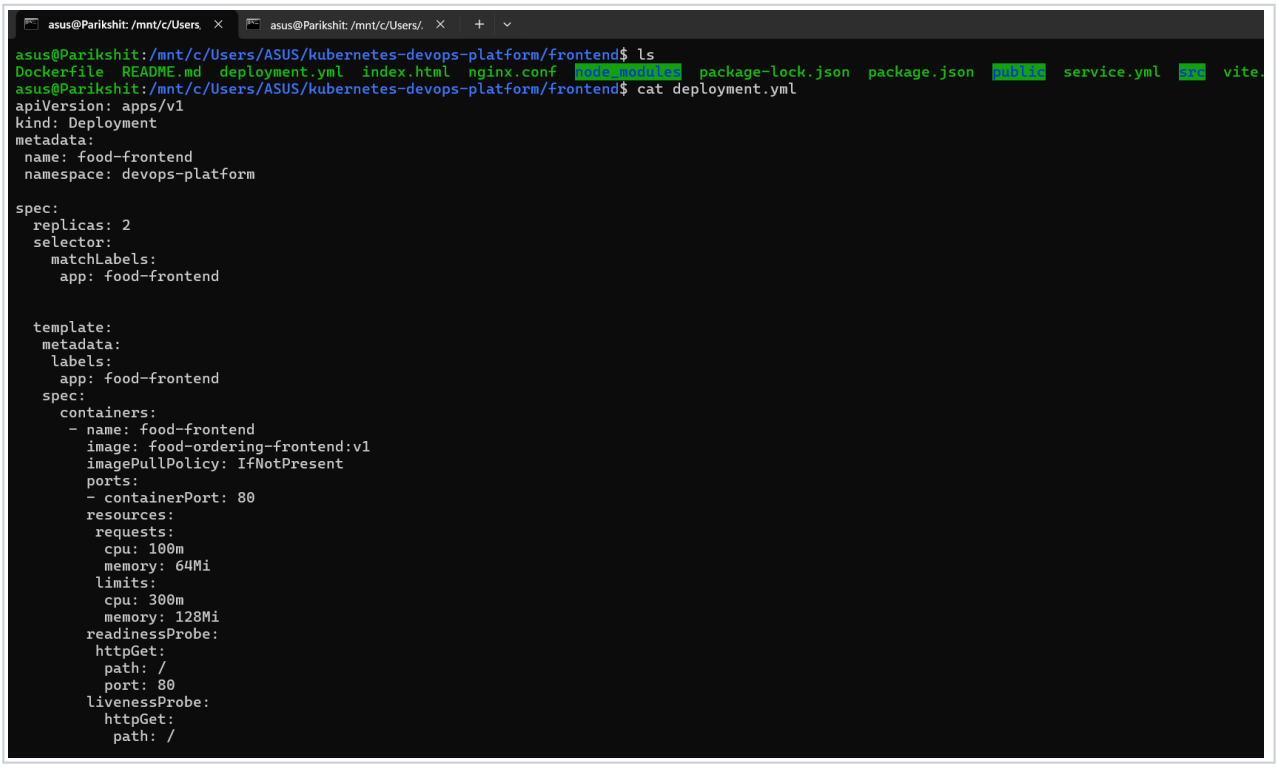


Figure 5.5: Frontend deployment.yml: 2 replicas, resource requests and limits, readiness and liveness probes.

5.4 Backend Helm Release

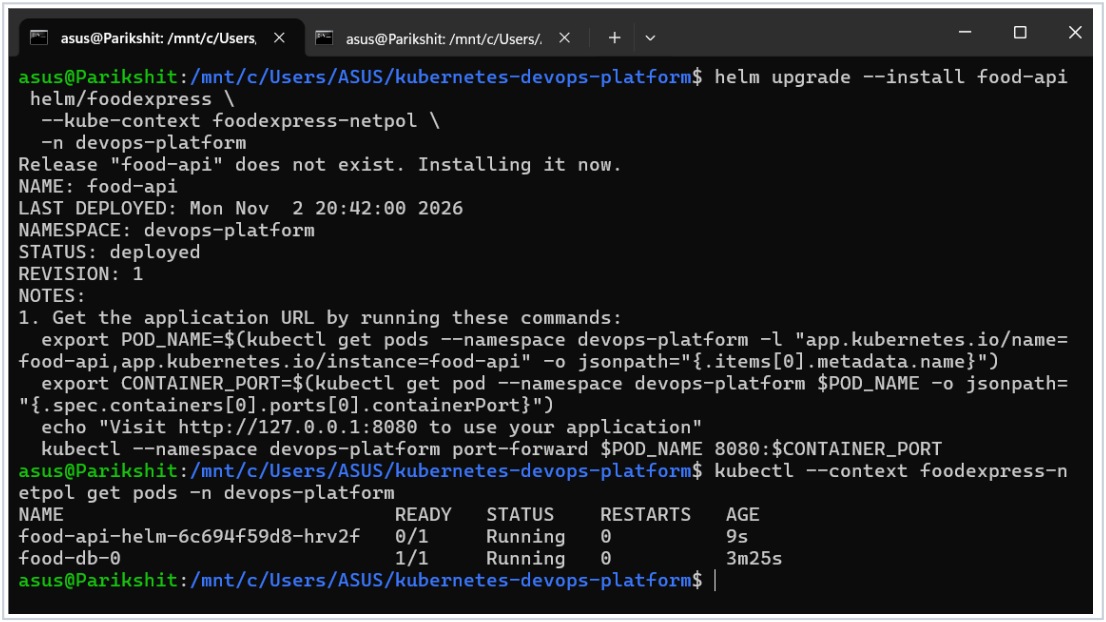


Figure 5.6: helm upgrade --install food-api on the foodexpress-netpol context, creating revision 1 and starting the backend pod.

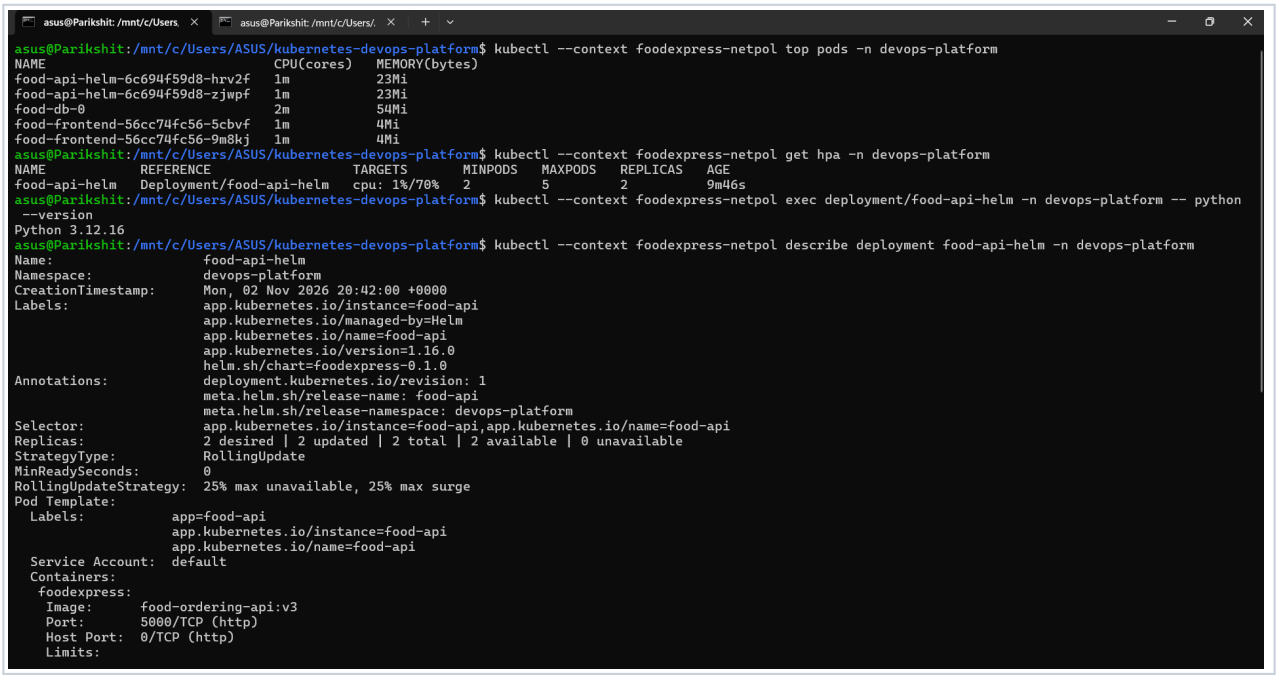


Figure 5.7: Backend Deployment: food-ordering-api:v3 on port 5000, RollingUpdate strategy and the default ServiceAccount.

5.5 Storage, Release, Autoscaler and Calico

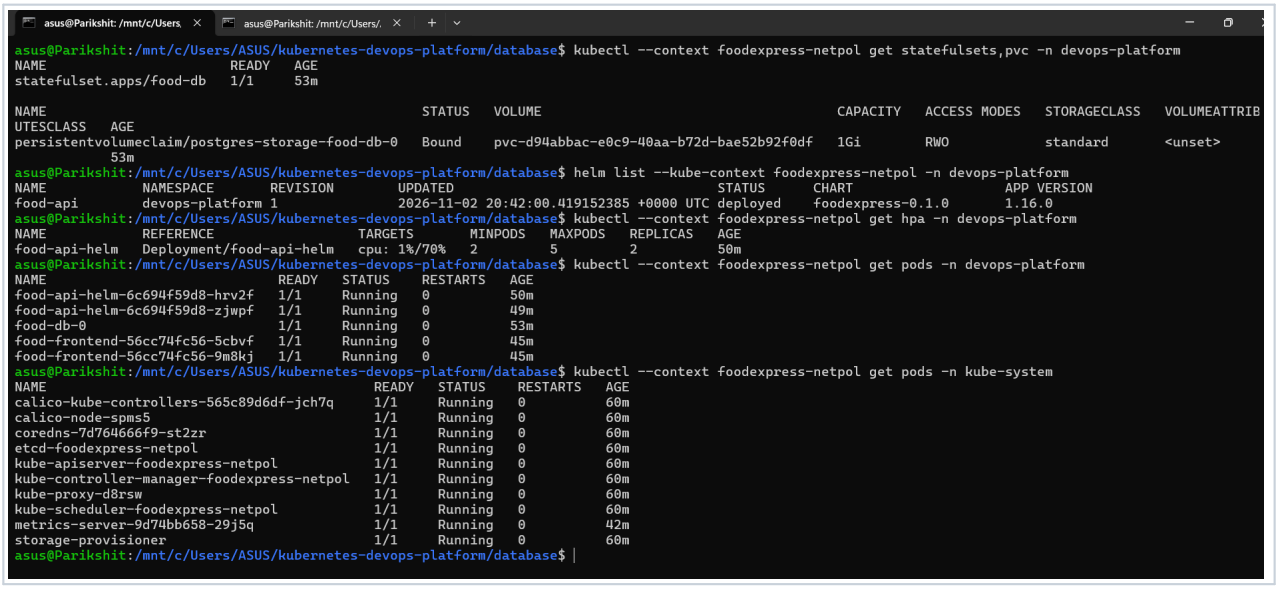


Figure 5.8: StatefulSet and PVC (Bound, 1Gi, RWO, standard), Helm release deployed, HPA at 1% of 70%, and calico-node and Metrics Server Running in kube-system.

The cluster system pods are all Running, including the two Calico components and `metrics-server`. The HPA reads 1% CPU against its 70% target, so the backend idles at its minimum of two replicas.

5.6 Frontend to Backend Connectivity

The backend health endpoint was called from inside a frontend pod using the Kubernetes Service name:

```
kubectl --context foodexpress-netpol exec deployment/food-frontend -n devops-platform -- \
  wget -qO- http://food-api-service:5000/health
# response: healthy
```

The successful response confirms that frontend pods reach the backend through `food-api-service`, and that the NetworkPolicy allows this path because the frontend pods carry the `app=food-frontend`

label.

5.7 Autoscaling Test

To test the HPA, a CPU-intensive Python loop was run inside the backend deployment for 120 seconds while the HPA was watched. The process ended when its 120-second alarm fired (exit code 142).

```
kubectl --context foodexpress-netpol exec deployment/food-api-helm -n devops-platform -- \
  python -c 'import time,signal; signal.alarm(120); x=0
while True: x+=1'
```

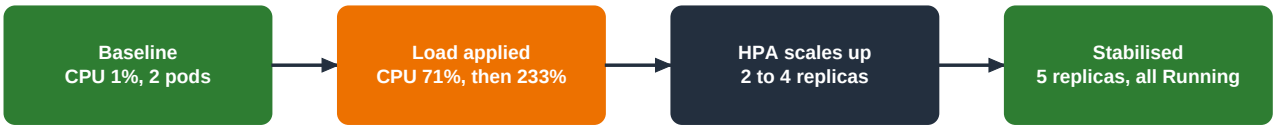


Figure 5.9: HPA scale-up sequence observed during the test

Observation	CPU (current / target)	Replicas
1. Baseline	1% / 70%	2
2. Load begins	71% / 70%	2
3. Peak utilization	233% / 70%	2
4. HPA reacts	233% / 70%	4
5. Final check	233% / 70%	5 (maximum)

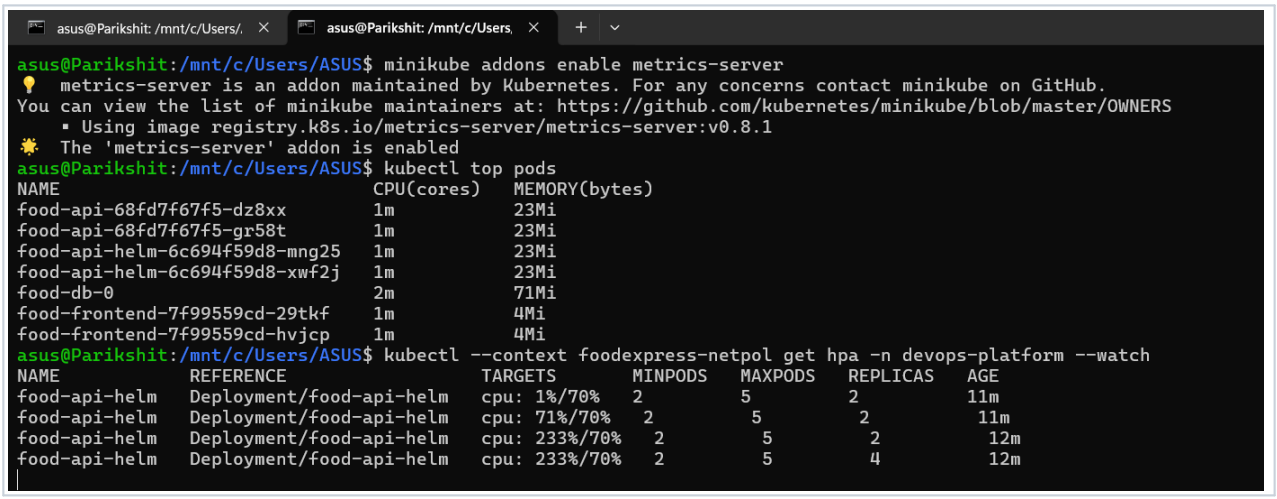


Figure 5.10: kubectl get hpa --watch: CPU climbs from 1% to 233% of the 70% target and replicas increase from 2 to 4.

```
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform$ kubectl --context foodexpress-n
etpol exec deployment/food-api-helm -n devops-platform -- python -c 'import time,signal; sig
nal.alarm(120); x=0
while True: x+=1'
command terminated with exit code 142
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform$ kubectl --context foodexpress-n
etpol get hpa -n devops-platform
NAME            REFERENCE                  TARGETS         MINPODS   MAXPODS   REPLICAS   AG
E
food-api-helm   Deployment/food-api-helm   cpu: 233%/70%   2         5         5          13
m
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform$ kubectl --context foodexpress-n
etpol get pods -n devops-platform
NAME                            READY   STATUS    RESTARTS   AGE
food-api-helm-6c694f59d8-4v9gh   1/1     Running   0          68s
food-api-helm-6c694f59d8-hrv2f   1/1     Running   0          13m
food-api-helm-6c694f59d8-nkt8k   1/1     Running   0          68s
food-api-helm-6c694f59d8-nvvdt   1/1     Running   0          53s
food-api-helm-6c694f59d8-zjwpf   1/1     Running   0          13m
food-db-0                        1/1     Running   0          16m
food-frontend-56cc74fc56-5cbvf   1/1     Running   0          9m30s
food-frontend-56cc74fc56-9m8kj   1/1     Running   0          9m28s
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform$
```

Figure 5.11: After the test the HPA reports 5 replicas and all five backend pods are Running with 0 restarts.

Result: CPU utilization exceeded the 70% target, reached about 233%, and the backend scaled from 2 to 5 replicas. All five pods reached the Running state and no restarts were observed during the reported check. A separate scale-down test was not performed.

5.8 NetworkPolicy, RBAC and Resource Usage

```
asus@Parikshit: /mnt/c/Users  asus@Parikshit: /mnt/c/Users/
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform/database$ kubectl --context foodexpress-netpol describe networkpolicy foodexpress-api-policy -n
devops-platform
Name:         foodexpress-api-policy
Namespace:    devops-platform
Created on:   2026-11-02 20:42:00 +0000 UTC
Labels:       app.kubernetes.io/managed-by=Helm
Annotations:  meta.helm.sh/release-name: food-api
              meta.helm.sh/release-namespace: devops-platform
Spec:
  PodSelector:     app=food-api
  Allowing ingress traffic:
    To Port: 5000/TCP
    From:
      PodSelector: app=food-frontend
  Not affecting egress traffic
  Policy Types: Ingress
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform/database$ kubectl --context foodexpress-netpol auth can-i get pods --as=system:serviceaccount:de
vops-platform:default -n devops-platform
yes
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform/database$ kubectl --context foodexpress-netpol top pods -n devops-platform
NAME                            CPU(cores)   MEMORY(bytes)
food-api-helm-6c694f59d8-hrv2f   1m           23Mi
food-api-helm-6c694f59d8-zjwpf   1m           23Mi
food-db-0                        1m           44Mi
food-frontend-56cc74fc56-5cbvf   1m           4Mi
food-frontend-56cc74fc56-9m8kj   1m           4Mi
asus@Parikshit:/mnt/c/Users/ASUS/kubernetes-devops-platform/database$
```

Figure 5.12: NetworkPolicy foodexpress-api-policy (ingress from app=food-frontend on 5000/TCP, egress not affected), an RBAC permission check returning yes, and pod CPU and memory usage.

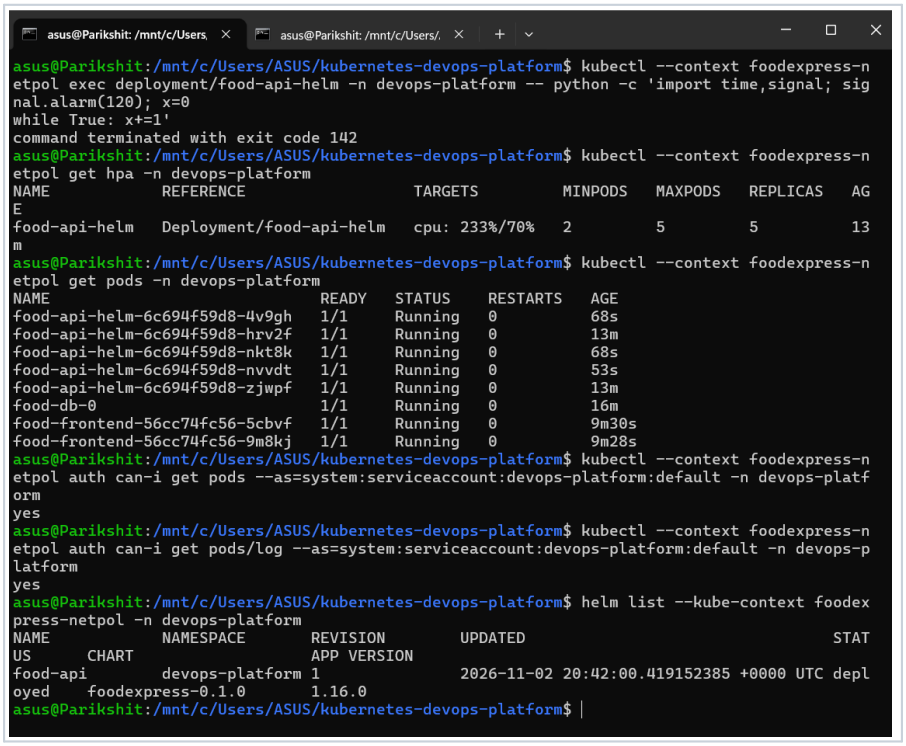


Figure 5.13: RBAC checks for get pods and get pods/log both return yes for the default ServiceAccount; Helm release food-api is deployed.

Pod	CPU	Memory
food-api-helm (each of 2 pods)	1m	23Mi
food-db-0	1m	44Mi
food-frontend (each of 2 pods)	1m	4Mi

Resource usage (from kubectl top pods) is low at idle, leaving plenty of headroom under the configured requests and limits.

6. Performance Testing with k6

6.1 Test Setup

k6 was used to load test the backend health endpoint. It was installed in the WSL Ubuntu environment (k6 v2.3.0) and run against the backend Service exposed locally by port forwarding. The port-forward must stay running in a separate terminal during the test.

```
k6 version
kubectl --context foodexpress-netpol port-forward svc/food-api-service 5001:5000 -n devops-platform
# test endpoint: http://localhost:5001/health
k6 run k6-test.js
```

6.2 Test Script

The script ramps virtual users in three stages and defines pass/fail thresholds for failures and latency.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 10 },
    { duration: '1m',  target: 20 },
    { duration: '30s', target: 0  },
  ],
  thresholds: {
    http_req_failed:   ['rate<0.01'],
    http_req_duration: ['p(95)<500'],
  },
};

export default function () {
  const health = http.get('http://localhost:5001/health');
  check(health, { 'API health status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

6.3 Load Profile

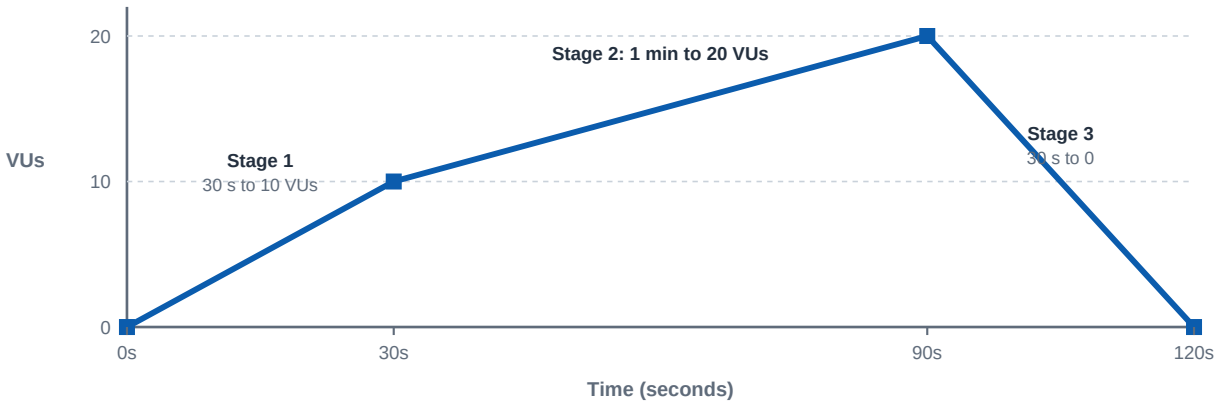


Figure 6.1: k6 virtual-user ramp: 0 to 10 VUs, 10 to 20 VUs, then ramp-down

Parameter	Configuration
Initial load	10 virtual users
Peak load / max virtual users	20
Ramp-up period	30 seconds
Peak duration	1 minute
Ramp-down period	30 seconds
p95 latency threshold	Less than 500 ms
HTTP failure threshold	Less than 1%

6.4 Results

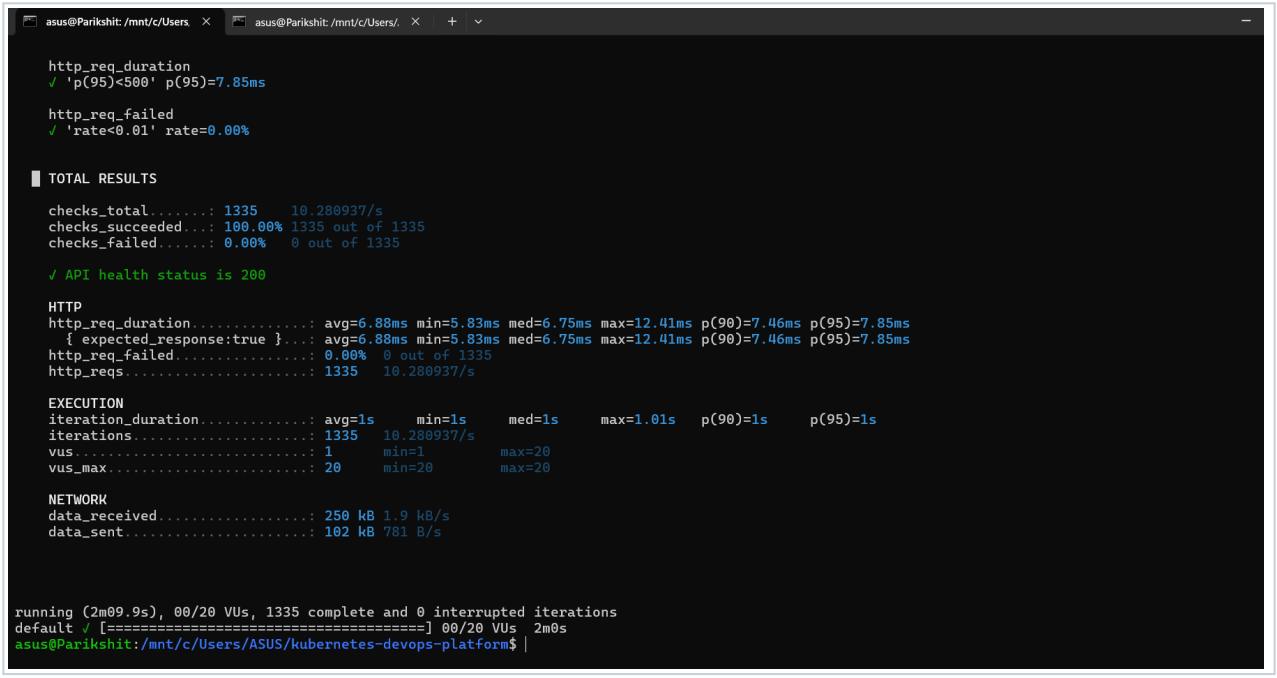


Figure 6.2: k6 summary: both thresholds pass, 1,335 of 1,335 checks succeeded, p95 = 7.85 ms, 0.00% request failures.

Metric	Actual result
Total HTTP requests	1,335 (10.28 per second)
Successful / failed checks	1,335 / 0
HTTP request failure rate	0.00%
Average response time	6.88 ms
Minimum / median / maximum	5.83 ms / 6.75 ms / 12.41 ms
90th percentile	7.46 ms
95th percentile	7.85 ms
Maximum virtual users	20
Data received / sent	250 kB / 102 kB
Test duration	About 2 minutes 10 seconds

6.5 Threshold Results

Threshold	Target	Actual	Result
http_req_duration	p(95) less than 500 ms	7.85 ms	PASS
http_req_failed	rate less than 1%	0.00%	PASS

6.6 Conclusion and Limitation

The k6 test completed successfully. All 1,335 requests passed the health check, and the observed p95 response time of 7.85 ms is far below the 500 ms threshold. No request failures were recorded.

Testing limitation
The test targets the **/health** endpoint only. It does not measure database queries, order creation, authentication or the complete food-ordering workflow, so it should not be read as a throughput figure for the whole application.

7. Troubleshooting

The following problems were met and resolved during implementation.

Issue	Cause	Resolution
NetworkPolicy not enforced	The original Minikube cluster had no suitable NetworkPolicy implementation.	Created a separate Minikube profile with the Calico CNI and kept the original cluster.
Backend container failing	An earlier backend image contained an application syntax error.	Built the corrected image food-ordering-api:v3 and loaded it into the dedicated cluster.
Frontend could not reach backend	The expected backend Service food-api-service did not exist.	Created a ClusterIP Service food-api-service on port 5000 for the Helm-managed pods and restarted the frontend Deployment.
HPA metrics unavailable	Metrics Server was not enabled, so CPU utilization could not be read.	Enabled the metrics-server addon in the dedicated Minikube profile.
k6 connection refused	The local port-forward to the backend was not running.	Restarted the port-forward, checked the endpoint with curl and reran k6 successfully.

7.1 Frontend Could Not Reach the Backend

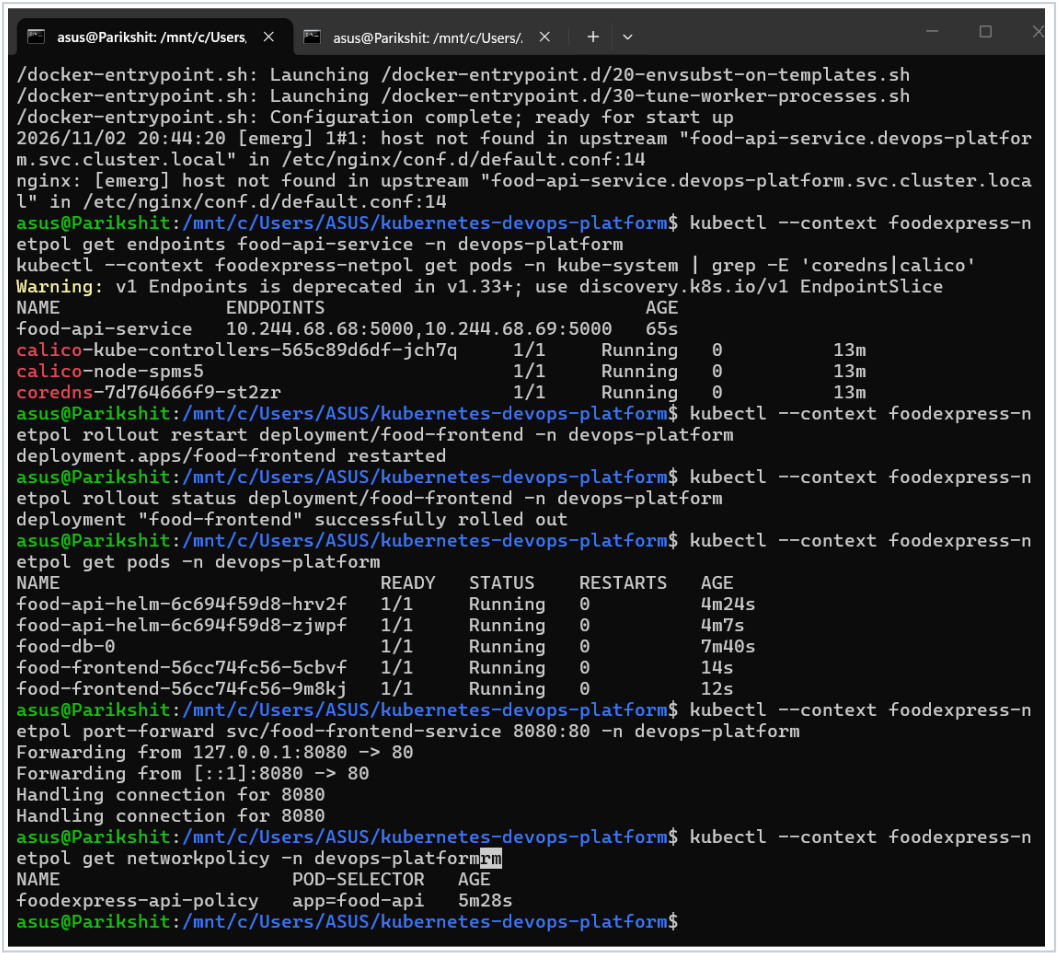


Figure 7.1: Frontend Nginx fails with host not found in upstream food-api-service; after the Service endpoints exist, the frontend Deployment is restarted and rolls out.

The frontend Nginx configuration proxies to `food-api-service.devops-platform.svc.cluster.local`. Until that Service existed, Nginx could not resolve the host and the container failed to start. After the Service was created, `kubectl get endpoints` listed both backend pod addresses on port 5000, and a rollout restart brought both frontend pods up. The same screenshot shows the NetworkPolicy listed afterwards.

7.2 k6 Connection Refused

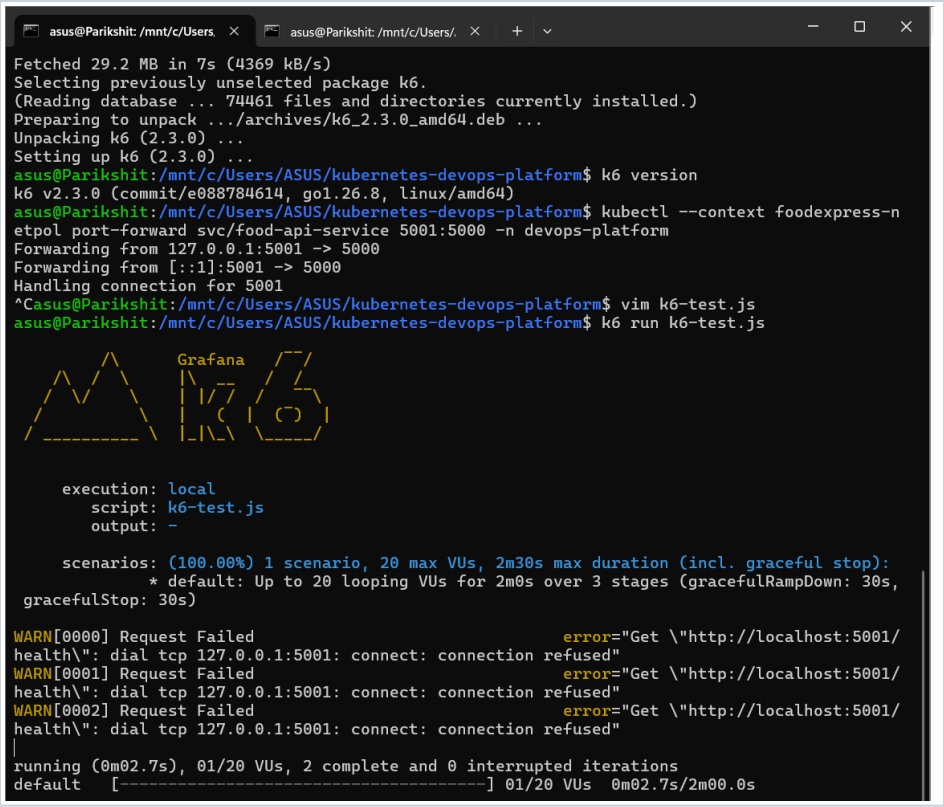


Figure 7.2: First k6 run failed with connection refused on 127.0.0.1:5001 because the port-forward process had been stopped.

8. Security Considerations and Implementation Summary

8.1 Security Controls

Area	Control in this project
Network access	Calico NetworkPolicy allows backend ingress only from frontend pods on TCP 5000.
Access control	RBAC Role limited to read-only access on pods and pod logs in one namespace.
Configuration	ConfigMap for settings, Secret for credentials. No passwords in documentation or images.
Resource governance	CPU and memory requests and limits on workloads so one pod cannot starve others.
Image handling	Locally built images loaded into the cluster, with multi-stage frontend build to keep the runtime image small.
Isolation	Dedicated namespace and a dedicated Minikube profile.

Kubernetes Secrets are only base64-encoded by default. For anything beyond a local environment, enable encryption at rest or use an external secrets manager.

8.2 Implementation Summary

Feature	Implementation	Status
Containerization	Docker images	Completed
Frontend deployment	React and Nginx	Completed
Backend deployment	Flask API	Completed
Database deployment	PostgreSQL StatefulSet	Completed
Persistent storage	1Gi PVC	Completed
Configuration management	ConfigMap and Secret	Completed
Package management	Helm	Completed
Container orchestration	Kubernetes	Completed
Automatic scaling	HPA	Scale-up tested
Metrics collection	Metrics Server	Verified
Container networking	Calico	Running
Network security	NetworkPolicy	Applied
Access control	RBAC	Permission checks passed
Application connectivity	Frontend to backend	Verified
Performance testing	k6	Passed

9. Limitations and Future Improvements

9.1 Current Limitations

- The platform runs on a single-node local Minikube cluster, not a managed or production cluster.
- The persistent volume is local and is not a backup or disaster-recovery solution.
- Only HPA scale-up was tested. Scale-down behaviour was not tested.
- The NetworkPolicy restricts ingress only; egress is unrestricted and the database has no dedicated policy.
- The RoleBinding uses the namespace default ServiceAccount.
- The load test covers the health endpoint only, not database operations or complete order workflows.
- The Service food-api-service was created manually, alongside the Helm-created backend Service.
- Images are loaded into Minikube directly and are not published to a registry.

9.2 Future Improvements

Improvement	Benefit
Dedicated ServiceAccount for the application	Least-privilege access and better isolation.
Egress policies and database-specific network rules	Moves toward a zero-trust network design.
Load test order creation and database operations	Realistic capacity figures for the full workflow.
Application metrics and centralized monitoring	Visibility into latency, errors and saturation.
CI/CD with GitHub Actions	Automated build, test and deployment.
Managed cloud Kubernetes (for example EKS)	Production-grade availability and scaling.
Production database backups and disaster recovery	Protects data beyond a local volume.
Publish images to a container registry	Repeatable, versioned image distribution.
TLS and ingress-based external access	Secure, standard external entry point.
Automated integration and security testing	Catches regressions and vulnerabilities early.
Move food-api-service into the Helm chart	Every backend resource is managed by one release.

10. Conclusion

FoodExpress demonstrates the deployment and management of a multi-tier application with Docker and Kubernetes. The project covers containerization, application deployment, Helm-based resource management, PostgreSQL StatefulSets, persistent storage, autoscaling, Calico NetworkPolicies, RBAC and performance testing.

The backend scaled from two to five replicas during a CPU load test, and the k6 health-endpoint test completed 1,335 successful HTTP checks with a 7.85 ms p95 response time and no failures. The troubleshooting record shows how each problem was diagnosed and fixed.

The project shows practical knowledge of Kubernetes administration, container orchestration, application networking, resource management, infrastructure security and performance testing. The listed future improvements are the natural next steps toward a production-ready deployment.

Project name	FoodExpress
Category	Kubernetes and DevOps Engineering
Environment	Local Minikube cluster
Dedicated cluster	foodexpress-netpol
Primary namespace	devops-platform

Appendix A: Project Directory Structure

```
kubernetes-devops-platform/
|
|-- backend/
|   |-- Dockerfile
|   |-- app.py
|   |-- configmap.yml
|   |-- deployment.yml
|   |-- pvc.yml
|   |-- requirements.txt
|   |-- secret.yml
|   `-- service.yml
|
|-- database/
|   `-- postgres.yml
|
|-- frontend/
|   |-- Dockerfile
|   |-- deployment.yml
|   |-- nginx.conf
|   `-- service.yml
|
|-- helm/
|   `-- foodexpress/
|       |-- Chart.yaml
|       |-- values.yaml
|       `-- templates/ (deployment, hpa, service, serviceaccount,
|                       networkpolicy, role, rolebinding, ingress,
|                       httproute, _helpers.tpl)
|
`-- k6-test.js
```

The repository also contains other folders (Figure A.1) that are not part of this listing. The listing shows the principal application and infrastructure files only.



Figure A.1: Top level of the project repository.

Appendix B: Useful Management Commands

All commands target the dedicated cluster and namespace.

```
# Everything in the namespace
kubectl --context foodexpress-netpol get all -n devops-platform

# Pods, Deployments, Services, storage
kubectl --context foodexpress-netpol get pods -n devops-platform
kubectl --context foodexpress-netpol get deployments -n devops-platform
kubectl --context foodexpress-netpol get services -n devops-platform
kubectl --context foodexpress-netpol get pvc -n devops-platform

# Autoscaling, policy and metrics
kubectl --context foodexpress-netpol get hpa -n devops-platform
kubectl --context foodexpress-netpol get networkpolicy -n devops-platform
kubectl --context foodexpress-netpol top pods -n devops-platform

# Helm
helm list --kube-context foodexpress-netpol -n devops-platform
```